

Requirement Engineering and Software Design

TTTE6104

Software Engineering
Paradigm

Objectives

- understand the concepts of user and system requirements and why these requirements should be written in different ways;
- understand the differences between functional and nonfunctional software requirements;
- understand how requirements may be organized in a software requirements document;
- understand the principal requirements engineering activities of elicitation, analysis and validation, and the relationships between these activities;
- understand why requirements management is necessary and how it supports other requirements engineering activities.

Contents

- Functional and non-functional requirements
- The software requirements document
- Requirements specification
- Requirements engineering processes
- Requirements elicitation and analysis
- Requirements validation
- Requirements management

Requirements

- The requirements for a system are the descriptions of what the system should do—the services that it provides and the constraints on its operation.
- Reflect the needs of customers for a system that serves a certain purpose such as controlling a device, placing an order, or finding information.
- Simply a high-level, abstract statement of a service that a system should provide or a constraint on a system, Or
- A detailed, formal definition of a system function

User requirements

- Statements, in a natural language plus diagrams, of what services the system is expected to provide to system users and the constraints under which it must operate.
- Write user requirements in natural language, with simple tables, forms, and intuitive diagrams.

System requirements

- more detailed descriptions of the software system's functions, services, and operational constraints.
- The system requirements document (sometimes called a functional specification) should define exactly what is to be implemented.
- It may be part of the contract between the system buyer and the software developers.

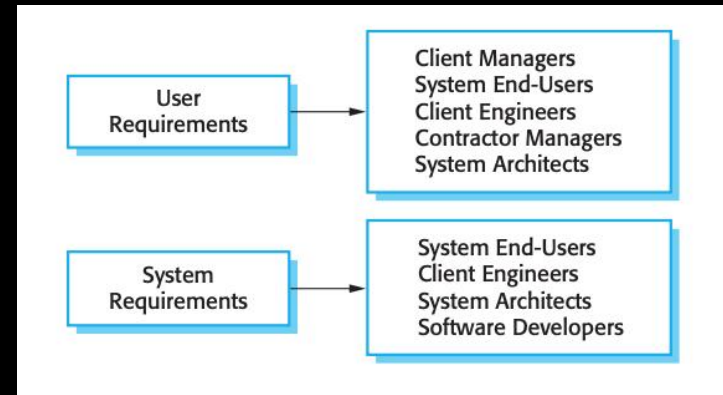
User and system requirements

User Requirement Definition

1. The MHC-PMS shall generate monthly management reports showing the cost of drugs prescribed by each clinic during that month.

System Requirements Specification

- 1.1 On the last working day of each month, a summary of the drugs prescribed, their cost, and the prescribing clinics shall be generated.
- 1.2 The system shall automatically generate the report for printing after 17.30 on the last working day of the month.
- 1.3 A report shall be created for each clinic and shall list the individual drug names, the total number of prescriptions, the number of doses prescribed, and the total cost of the prescribed drugs.
- 1.4 If drugs are available in different dose units (e.g., 10 mg, 20 mg) separate reports shall be created for each dose unit.
- 1.5 Access to all cost reports shall be restricted to authorized users listed on a management access control list.



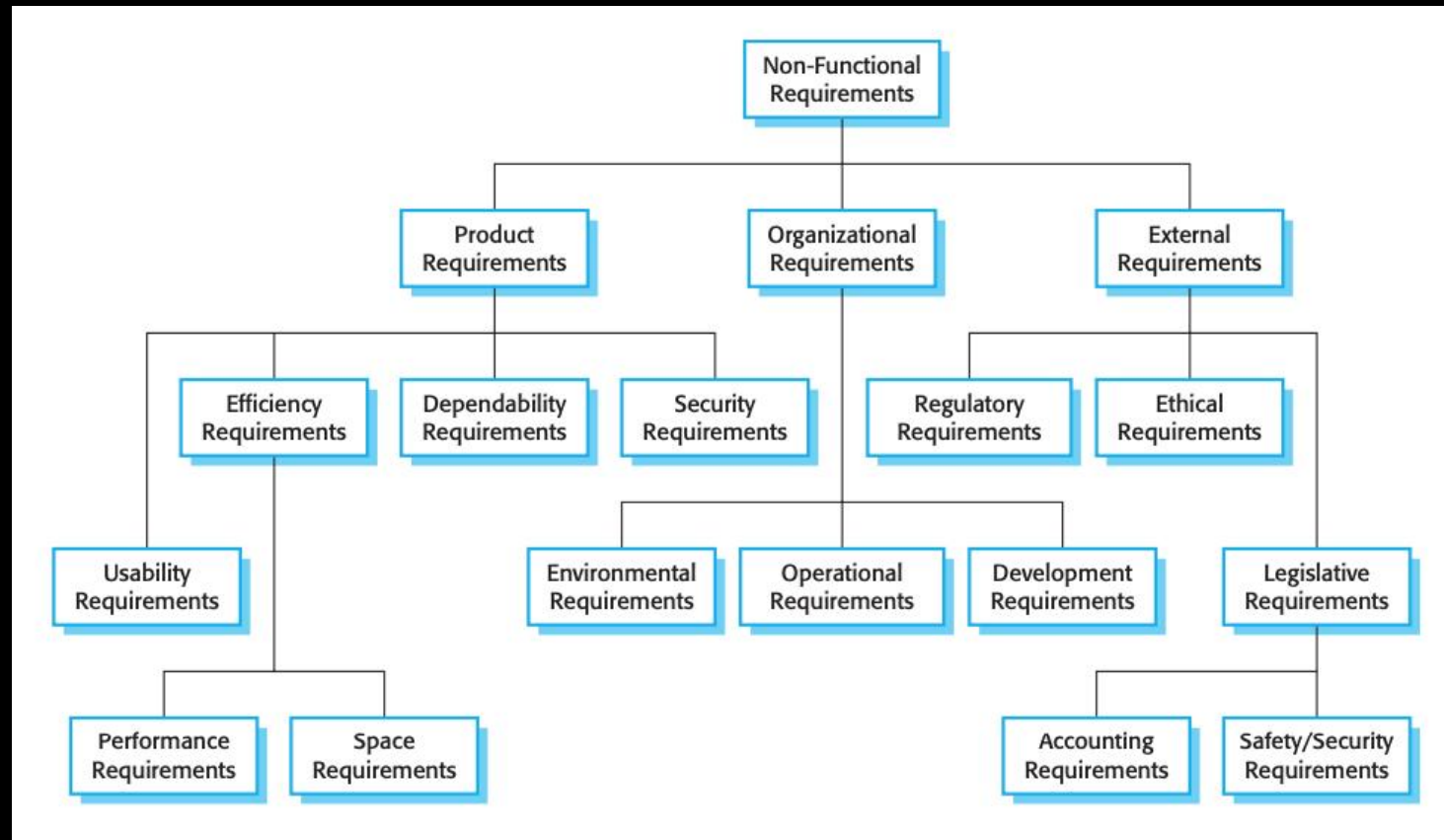
Requirements Engineering (RE)

- The process of finding out, analyzing, documenting and checking these services and constraints.

Functional and non-functional requirements

- Functional requirements
 - Statements of services the system should provide, how the system should react to particular inputs, and how the system should behave in particular situations.
 - May also explicitly state what the system should not do.
- Non-functional requirements
 - These are constraints on the services or functions offered by the system. They include timing constraints, constraints on the development process, and constraints imposed by standards.
 - Often apply to the system as a whole, rather than individual system features or services.

Types of non-functional requirement

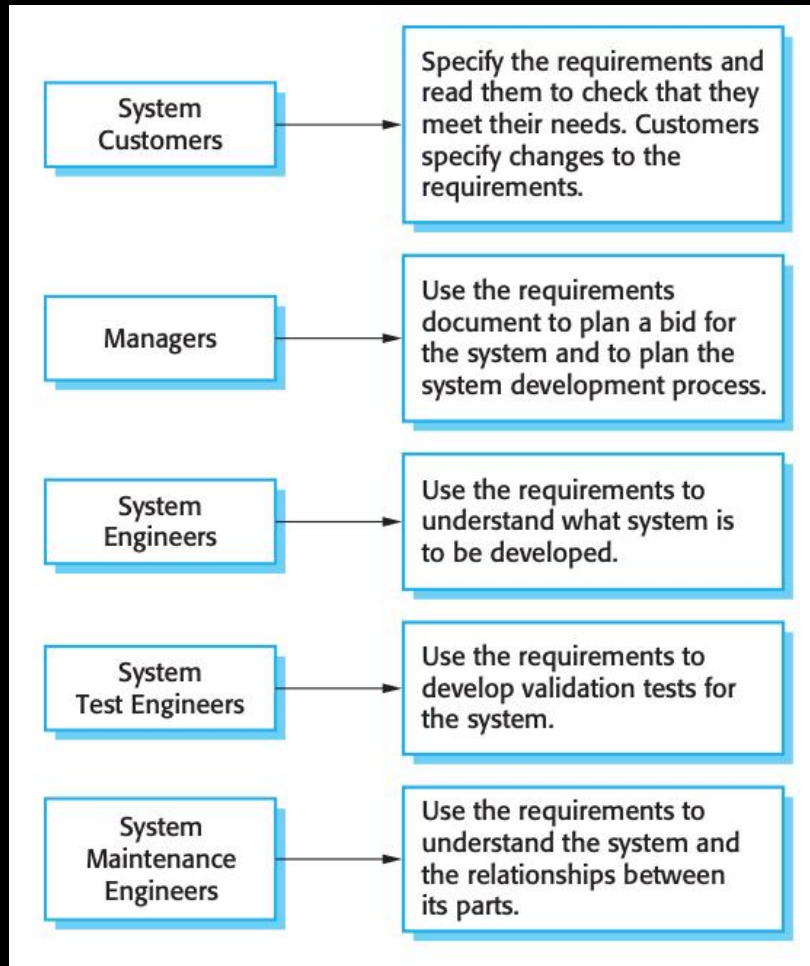


Class Activity: Identifying Functional and Non-Functional Requirements

- **Form groups of 3–4 students.** Select one of the following real-world systems: Online shopping platform (e.g., Amazon, Lazada)
 - Food delivery app (e.g., Grab, UberEats)
 - Online banking system (e.g., Maybank2u, CIMB Clicks)
 - Ride-hailing app (e.g., Grab, Lyft)
- For your chosen system, **identify**:
 - **2 Functional requirements:** Describe services the system should provide or how it should react in certain situations.
 - **2 Non-functional requirements:** Describe constraints related to performance, security, usability, etc., that apply to the system.
- **Prepare a brief 5-minute presentation** to share your examples with the class.

The software requirements document

- Sometimes called the software requirements specification or SRS
- An official statement of what the system developers should implement.
- It should include both the user requirements for a system and a detailed specification of the system requirements.
- Essential when an outside contractor is developing the software system.



Users of a requirements document

The structure of a requirements document

Chapter	Description
Preface	This should define the expected readership of the document and describe its version history, including a rationale for the creation of a new version and a summary of the changes made in each version.
Introduction	This should describe the need for the system. It should briefly describe the system's functions and explain how it will work with other systems. It should also describe how the system fits into the overall business or strategic objectives of the organization commissioning the software.
Glossary	This should define the technical terms used in the document. You should not make assumptions about the experience or expertise of the reader.
User requirements definition	Here, you describe the services provided for the user. The non-functional system requirements should also be described in this section. This description may use natural language, diagrams, or other notations that are understandable to customers. Product and process standards that must be followed should be specified.
System architecture	This chapter should present a high-level overview of the anticipated system architecture, showing the distribution of functions across system modules. Architectural components that are reused should be highlighted.
System requirements specification	This should describe the functional and non-functional requirements in more detail. If necessary, further detail may also be added to the non-functional requirements. Interfaces to other systems may be defined.
System models	This might include graphical system models showing the relationships between the system components, the system, and its environment. Examples of possible models are object models, data-flow models, or semantic data models.
System evolution	This should describe the fundamental assumptions on which the system is based, and any anticipated changes due to hardware evolution, changing user needs, and so on. This section is useful for system designers as it may help them avoid design decisions that would constrain likely future changes to the system.
Appendices	These should provide detailed, specific information that is related to the application being developed; for example, hardware and database descriptions. Hardware requirements define the minimal and optimal configurations for the system. Database requirements define the logical organization of the data used by the system and the relationships between data.
Index	Several indexes to the document may be included. As well as a normal alphabetic index, there may be an index of diagrams, an index of functions, and so on.

Requirements specification

- Process of writing down the user and system requirements in a requirements document.
- Should be clear, unambiguous, easy to understand, complete, and consistent.
- Should not include details of the system architecture or design
- User requirements – describe the functional and non– functional requirements so that they are understandable by system users who don't have detailed technical knowledge
- System requirements – describe the external behavior of the system and its operational constraints.
-

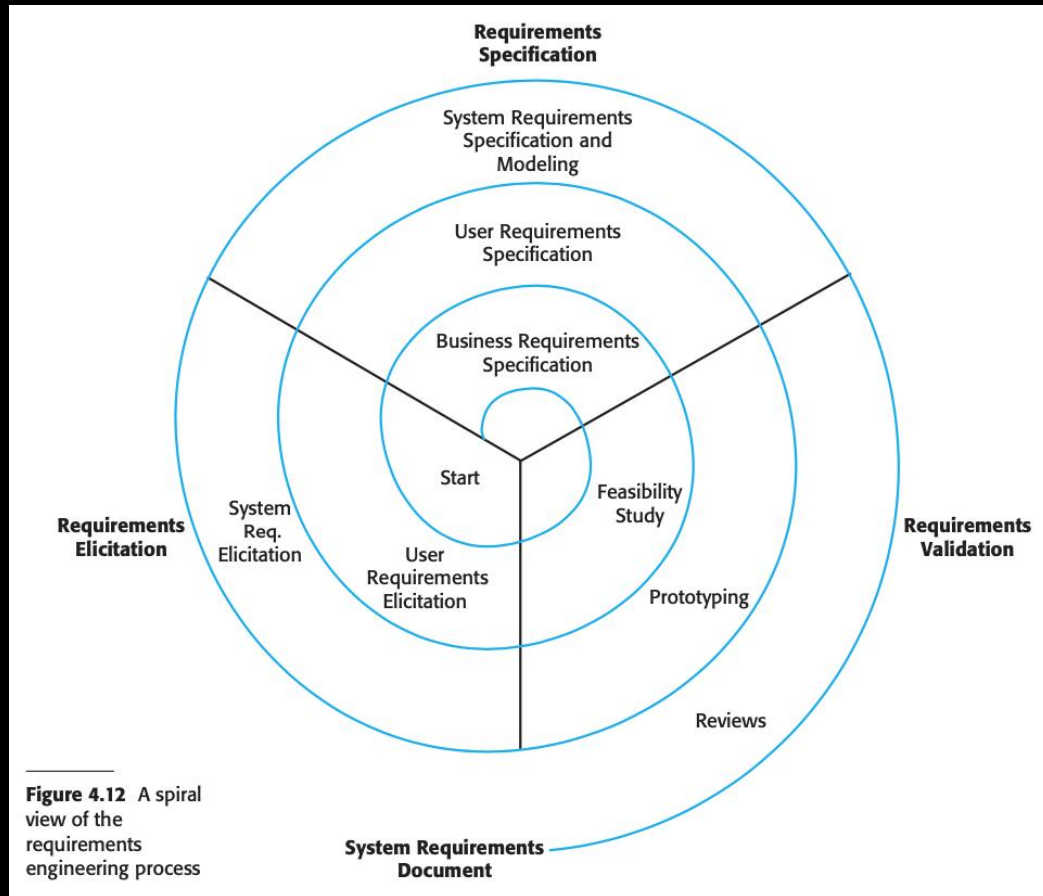
Ways of writing a system requirements specification

Notation	Description
Natural language sentences	The requirements are written using numbered sentences in natural language. Each sentence should express one requirement.
Structured natural language	The requirements are written in natural language on a standard form or template. Each field provides information about an aspect of the requirement.
Design description languages	This approach uses a language like a programming language, but with more abstract features to specify the requirements by defining an operational model of the system. This approach is now rarely used although it can be useful for interface specifications.
Graphical notations	Graphical models, supplemented by text annotations, are used to define the functional requirements for the system; UML use case and sequence diagrams are commonly used.
Mathematical specifications	These notations are based on mathematical concepts such as finite-state machines or sets. Although these unambiguous specifications can reduce the ambiguity in a requirements document, most customers don't understand a formal specification. They cannot check that it represents what they want and are reluctant to accept it as a system contract.

A spiral view of the requirements engineering process

Discussion Question:

In the spiral view of the requirements engineering process, requirements are gathered and refined through multiple iterations. Can you explain how this spiral model works and why it is beneficial for managing requirements in complex software projects? What are the key stages involved in each iteration?



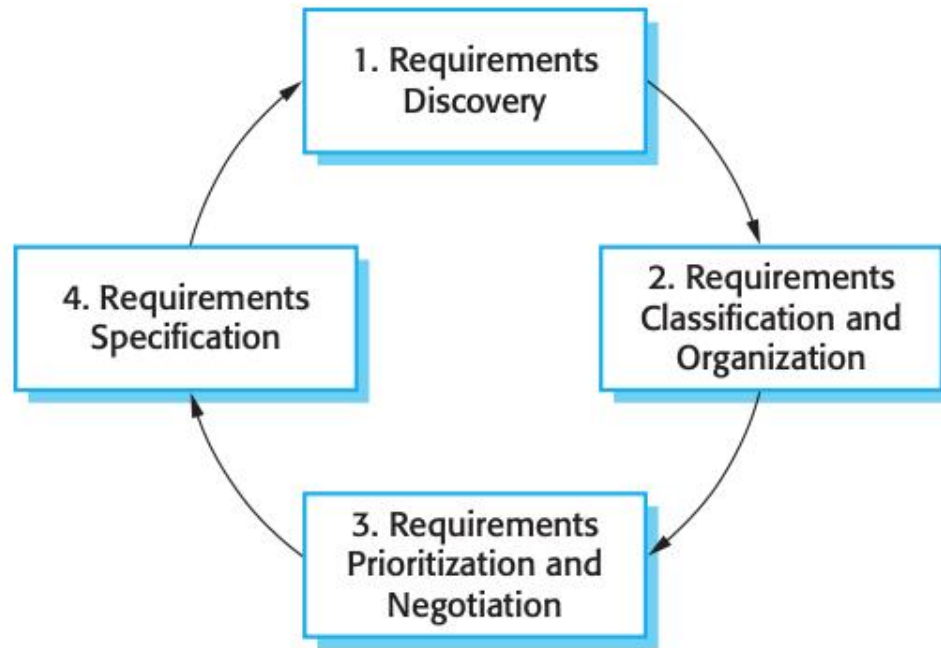
Requirements engineering processes

- Include four high-level activities.
- Feasibility study – Focus on assessing if the system is useful to the business
- elicitation and analysis – Discovering requirements
- Specification – Converting these requirements into some standard form
- Validation – Checking that the requirements define the system that the customer wants

Requirements elicitation and analysis

- Software engineers work with customers and system end-users to find out about the application domain, what services the system should provide, the required performance of the system, hardware constraints, and so on.
- The process activities are:
 - Requirements discovery
 - Requirements classification and organization
 - Requirements prioritization and negotiation
 - Requirements specification

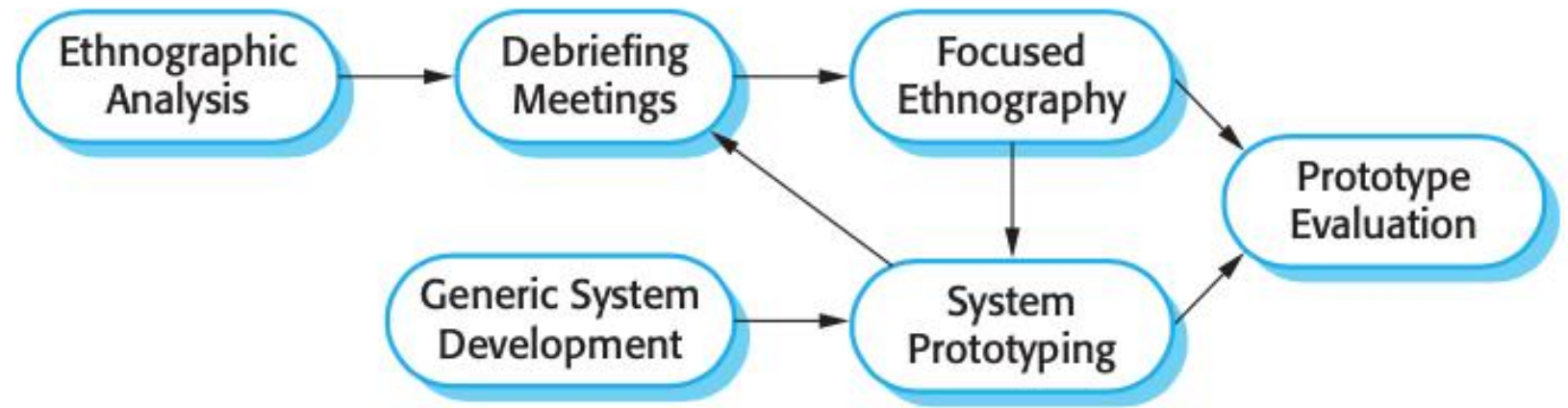
The requirements elicitation and analysis process



Iterative process

Techniques for Requirements Discovery

- Interviews
- Scenarios – starts with an outline of the interaction. During the elicitation process, details are added to this to create a complete description of that interaction.
- Use cases – a fundamental feature of the unified modeling language
- Ethnography – an observational technique that can be used to understand operational processes and help derive support requirements for these processes.



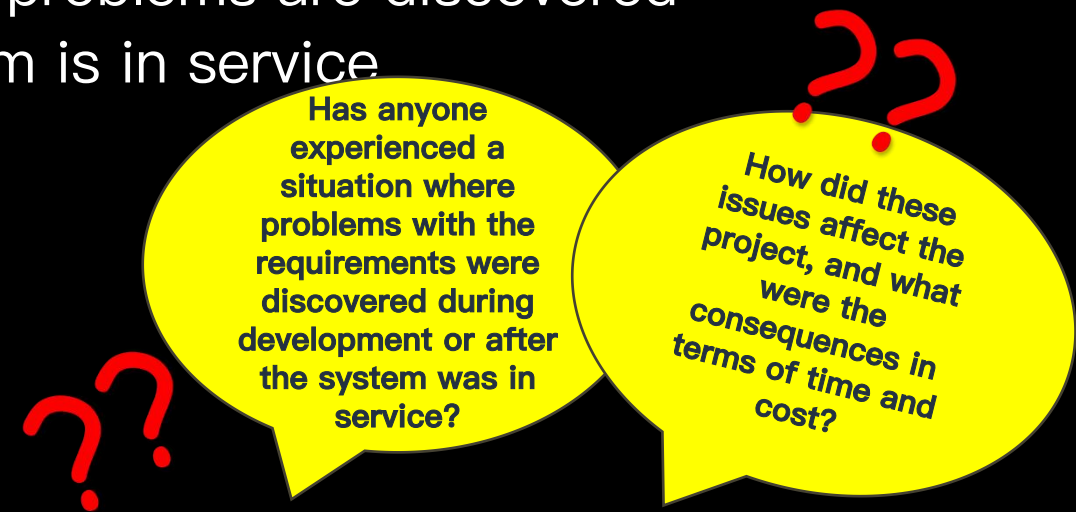
Ethnography and prototyping for requirements analysis

Eliciting and understanding requirements from system stakeholders is a difficult process for several reasons

- Stakeholders often don't know what they want from a computer system except in the most general terms; they may find it difficult to articulate what they want the system to do; they may make unrealistic demands because they don't know what is and isn't feasible.
- Stakeholders in a system naturally express requirements in their own terms and with implicit knowledge of their own work. Requirements engineers, without experience in the customer's domain, may not understand these requirements.
- Different stakeholders have different requirements and they may express these in different ways. Requirements engineers have to discover all potential sources of requirements and discover commonalities and conflict.
- Political factors may influence the requirements of a system. Managers may demand specific system requirements because these will allow them to increase their influence in the organization.
- The economic and business environment in which the analysis takes place is dynamic. It inevitably changes during the analysis process. The importance of particular requirements may change. New requirements may emerge from new stakeholders who were not originally consulted.

Requirements validation

- Process of checking that requirements actually define the system that the customer really wants
- It is concerned with finding problems with the requirements.
- It is important because errors in a requirements document can lead to extensive rework costs when these problems are discovered during development or after the system is in service



Different types of checks

- Validity checks – Further thought and analysis may identify additional or different functions that are required. Systems have diverse stakeholders with different needs.
- Consistency checks – Requirements in the document should not conflict. There should not be contradictory constraints or different descriptions of the same system function.
- Completeness checks – The requirements document should include requirements that define all functions and the constraints intended by the system user.
- Realism checks – Using knowledge of existing technology, the requirements should be checked to ensure that they can actually be implemented.
- Verifiability – To reduce the potential for dispute between customer and contractor, system requirements should always be written so that they are verifiable. Write a set of tests that can demonstrate that the delivered system meets each specified requirement.

Requirements Validation Techniques

- Requirements reviews – The requirements are analyzed systematically by a team of reviewers who check for errors and inconsistencies.
- Prototyping – In this approach to validation, an executable model of the system in question is demonstrated to end-users and customers. They can experiment with this model to see if it meets their real needs.
- Test-case generation – Requirements should be testable. If the tests for the requirements are devised as part of the validation process, this often reveals requirements problems. If a test is difficult or impossible to design, this usually means that the requirements will be difficult to implement and should be reconsidered.

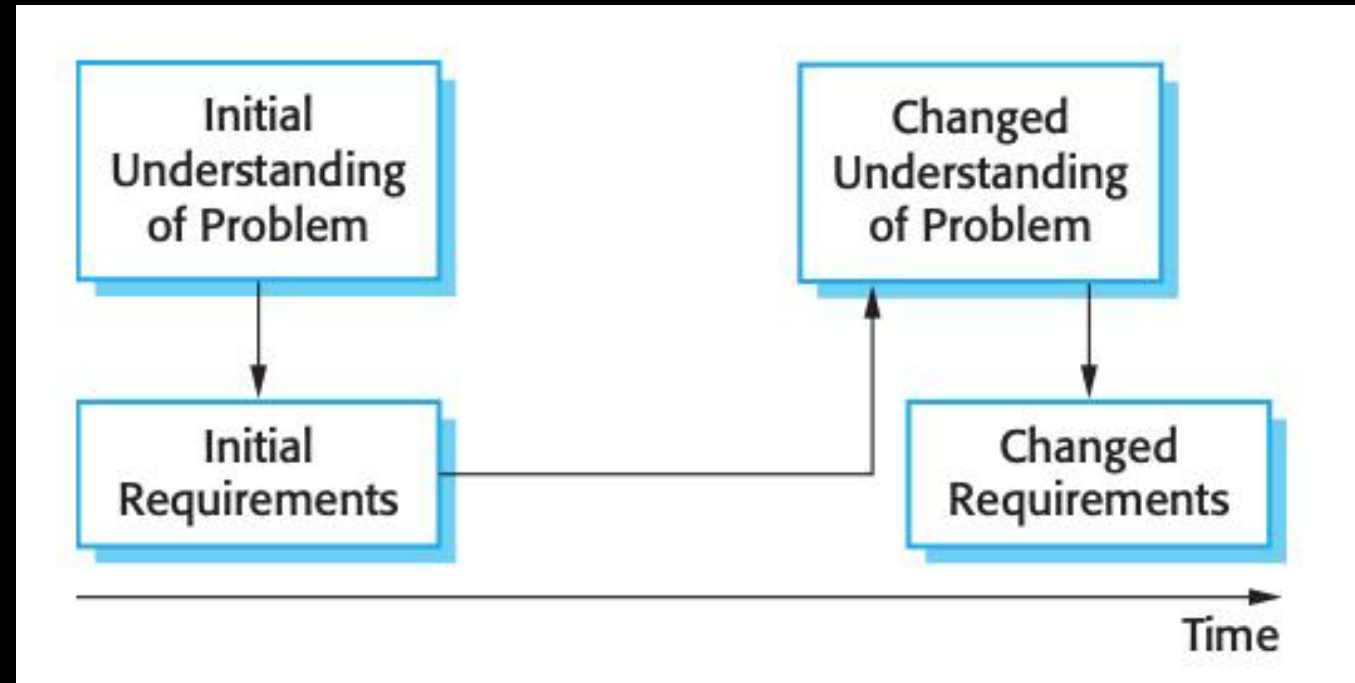
Class Activity: Researching Real-World Applications of Requirements Validation Techniques

Instructions:

1. Research and find a real-world case study, report, or article that demonstrates the use of one of the following requirements validation techniques:
 1. **Requirements Reviews**
 2. **Prototyping**
 3. **Test-case Generation**
 2. In your research, focus on how the chosen technique was applied to a software development project and identify:
 1. The system or project being validated.
 2. The process followed in the application of the technique.
 3. The challenges faced or benefits gained from using the technique.
 4. Any issues found with the requirements and how they were resolved.
 3. **Summarize the findings:**
 1. Briefly explain the case to the class in a **3-minute presentation**.
 2. Share how this technique helped or hindered the success of the project.
- **Example Sources:**
 - Software project case studies from journals or reputable websites.
 - White papers or technical reports from software companies.
 - Blogs or articles detailing experiences with requirements validation in real projects.

Requirements management

- The requirements for large software systems are always changing.
- The stakeholders' understanding of the problem is constantly changing because:
 - systems are usually developed to address problems that cannot be completely defined
 - the software requirements are bound to be incomplete
- The system requirements must then also evolve to reflect this changed problem view.



Requirements evolution

There are several reasons why change is inevitable

- The business and technical environment of the system always changes after installation. New hardware may be introduced, it may be necessary to interface the system with other systems, business priorities may change (with consequent changes in the system support required), and new legislation and regulations may be introduced that the system must necessarily abide by.
- The people who pay for a system and the users of that system are rarely the same people. System customers impose requirements because of organizational and budgetary constraints. These may conflict with end-user requirements and, after delivery, new features may have to be added for user support if the system is to meet its goals.
- Large systems usually have a diverse user community, with many users having different requirements and priorities that may be conflicting or contradictory. The final system requirements are inevitably a compromise between them and, with experience, it is often discovered that the balance of support given to different users has to be changed.

Requirements management planning

- Requirements identification – Each requirement must be uniquely identified so that it can be cross-referenced with other requirements and used in traceability assessments.
- A change management process – This is the set of activities that assess the impact and cost of changes. I discuss this process in more detail in the following section.
- Traceability policies – These policies define the relationships between each requirement and between the requirements and the system design that should be recorded. The traceability policy should also define how these records should be maintained.
- Tool support – Requirements management involves the processing of large amounts of information about the requirements. Tools that may be used range from specialist requirements management systems to spreadsheets and simple database systems.

Requirements change management

- Should be applied to all proposed changes to a system's requirements after the requirements document has been approved.
- Change management is essential because you need to decide if the benefits of implementing new requirements are justified by the costs of implementation.

Three principal stages to a change management process

- Problem analysis and change specification – The process starts with an identified requirements problem. During this stage, the problem or the change proposal is analyzed to check that it is valid.
- Change analysis and costing – The effect of the proposed change is assessed using traceability information and general knowledge of the system requirements. The cost of making the change is estimated both in terms of modifications to the requirements document and, if appropriate, to the system design and implementation.
- Change implementation – The requirements document and, where necessary, the system design and implementation, are modified.

KEY POINTS

- Requirements for a software system set out what the system should do and define constraints on its operation and implementation.
- Functional requirements are statements of the services that the system must provide or are descriptions of how some computations must be carried out.
- Non-functional requirements often constrain the system being developed and the development process being used. These might be product requirements, organizational requirements, or external requirements. They often relate to the emergent properties of the system and therefore apply to the system as a whole.
- The software requirements document is an agreed statement of the system requirements. It should be organized so that both system customers and software developers can use it.
- The requirements engineering process includes a feasibility study, requirements elicitation and analysis, requirements specification, requirements validation, and requirements management.
- Requirements elicitation and analysis is an iterative process that can be represented as a spiral of activities—requirements discovery, requirements classification and organization, requirements negotiation, and requirements documentation.
- Requirements validation is the process of checking the requirements for validity, consistency, completeness, realism, and verifiability.
- Business, organizational, and technical changes inevitably lead to changes to the requirements for a software system. Requirements management is the process of managing and controlling these changes.

From Requirements to System Design

- How the validated requirements are translated into a system architecture and design?
- System design is the process of defining the architecture, components, modules, interfaces, and data for a system to satisfy specified requirements.

Key Activities in System Design

- Architectural Design
 - Defines the high-level structure of the system, identifying major components and how they interact.
 - Common architectural styles: Client-server, Layered, Microservices, etc.
- Component Design
 - Decomposes the system into smaller, independent modules or components.
 - Defines the responsibilities and interfaces of each component.
- Data Design
 - Defines how data will be structured, stored, and accessed.
 - Includes designing databases, data flows, and data models.
 - Ensures data integrity, consistency, and security.
- Interface Design
 - Defines how system components will communicate with each other.
 - Ensures seamless integration between system modules and a user-friendly interface.

Class Activity : Design Principles and Best Practices

- Explore the following design principles or best practices:
 - Separation of Concerns
 - Modularity
 - Abstraction
 - Encapsulation
 - Design for Scalability
- Discuss your assigned principle or best practice. Consider the following points:
 - What does the principle mean in the context of software design?
 - Why is it important for building effective and maintainable software systems?
 - Provide a real-world example or case study that illustrates its use.
- Give a brief presentation to the class.

Class Activity: Identifying Design Models and Tools

- Research and identify one design model (e.g., class diagram, sequence diagram, use case diagram) and one tool (e.g., Visual Paradigm, Lucidchart, Microsoft Visio) commonly used in system design.
- For each, provide:
 - A brief description of what the design model or tool is used for.
 - An example of how it could be applied in a real-world software project.
- Prepare a 2-minute explanation of your findings to share with the class.

References

- Sommerville, I. (2016). Software engineering. Pearson.